UNIVERSITY OF TECHNOLOGY SYDNEY

Demonstration Practical 2
State-Feedback Control

Deadline: Sunday 18th October 2026.

Q1. State-Space Model (40 marks)

Q1.1) System Modelling: Obtain a state-space model for your final project setup:

$$\dot{\boldsymbol{x}}(t) = \boldsymbol{A}_c\boldsymbol{x}(t) + \boldsymbol{B}_c\boldsymbol{u}(t), \tag{1a}$$
$$\boldsymbol{y}(t) = \boldsymbol{C}_c\boldsymbol{x}(t) + \boldsymbol{D}_c\boldsymbol{u}(t). \tag{1b}$$

This state-space model can be derived using first principles (e.g., Newton's second law of motion, Kirchhoff's circuit laws, the principle of least action, etc.). Clearly specify the number of system states n, inputs m, and outputs p. Moreover, provide the continuous-time matrices $\boldsymbol{A}_c$, $\boldsymbol{B}_c$, $\boldsymbol{C}_c$, and $\boldsymbol{D}_c$. All matrices must be rigorously derived.

Remark 1. *If the analytical model is nonlinear, obtain the state-space matrices using Jacobian linearisation around a specific equilibrium (operating) point.*

Q1.2) System Parameters: Provide the physical parameter values (e.g., mass, length, moment of inertia, friction coefficient, resistance, inductance, etc.) used in your project. If these parameters are not explicitly provided, perform system identification in MATLAB using experimental input-output data from your setup.

Q1.3) System Model Discretisation: Select a proper sampling-time and then obtain a discrete-time state-space model version for your system:

$$\boldsymbol{x}(k+1) = \boldsymbol{A}\boldsymbol{x}(k) + \boldsymbol{B}\boldsymbol{u}(k), \tag{2a}$$
$$\boldsymbol{y}(k) = \boldsymbol{C}\boldsymbol{x}(k) + \boldsymbol{D}\boldsymbol{u}(k). \tag{2b}$$

Q2. State-Feedback Control (SFC) (40 marks)

After obtaining a state-space model for your system, design a state-feedback control system for your setup. Ensure this control system satisfies the performance requirements specified for your project on Canvas. Use the provided `main_SFC.m` file to help you with the requires steps to design your SFC.

Q2.1) Controllability: Provide the controllability matrix for the continuous-time state-space model and determine whether the system is controllable. Properly justify your answer.

Q2.2) State and Input References (Equilibrium Point): Given the desired output reference

vector:

$$\boldsymbol{y}^\star = \begin{bmatrix} y_1^\star \\ \vdots \\ y_p^\star \end{bmatrix}, \tag{3}$$

determine the corresponding steady-state references for the system state vector, $\boldsymbol{x}^\star$, and the control input vector, $\boldsymbol{u}^\star$.

Q2.3) Closed-Loop Eigenvalues: Based on the control requirements provided on Canvas for your setup, determine the n desired continuous- and discrete-time closed-loop eigenvalues (poles).

Q2.4) State-Feedback Gain: Obtain the state-feedback gain, $\boldsymbol{F}$, for the control law:

$$\boldsymbol{u}(k) = -\boldsymbol{F}\left(\boldsymbol{x}(k) - \boldsymbol{x}^\star\right) + \boldsymbol{u}^\star. \tag{4}$$

Verify that the closed-loop system matrix $(\boldsymbol{A} - \boldsymbol{B}\boldsymbol{F})$ has the desired closed-loop eigenvalues calculated in Q2.3).

Q2.5) Closed-Loop Implementation: In Simulink, implement your designed state-feedback control law from Q2.4) using a MATLAB Function block. At this stage, assume that full state measurement is available. For nonlinear systems, apply your controller to both the linearised and nonlinear system models. Provide relevant plots to demonstrate that the closed-loop system meets the control requirements specified on Canvas.

Remark 2. *Set* `run_observer = 0;` *in your* `main_SFC.m` *file to only run the SFC.*

Remark 3. *Do not provide screenshots of the Simulink scopes. Instead, export the data to the workspace and plot the signals using the* `plots_Linear_case.m` *or* `plots_Nonlinear_case.m` *files to create your plots. Add labels and legends.*

Q3. Luenberger Observer Design (20 marks)

In practice, not all states of the physical system can be directly measured. Therefore, an observer must be designed to estimate the full state vector from the available inputs and measured outputs.

Q3.1) Observability: Provide the observability matrix for the continuous-time state-space model and determine whether the system is observable. Properly justify your answer.

Q3.2) Observer Eigenvalues: Based on the control requirements provided on Canvas for your setup, determine the n desired continuous- and discrete-time observer eigenvalues.

Q3.3) Observer Gain: Obtain the observer gain, $\boldsymbol{L}$, for the discrete-time estimator:

$$\hat{\boldsymbol{x}}(k+1) = \boldsymbol{A}\hat{\boldsymbol{x}}(k) + \boldsymbol{B}\boldsymbol{u}(k) + \boldsymbol{L}\left(\boldsymbol{y}(k) - \hat{\boldsymbol{y}}(k)\right) \tag{5}$$

Verify that the observer error dynamics matrix $(\boldsymbol{A} - \boldsymbol{L}\boldsymbol{C})$ has the desired observer eigenvalues calculated in Q3.2).

Q3.4) Closed-Loop Implementation: In Simulink, implement your designed Luenberger observer from Q3.3) alongside your state-feedback controller from Q2.4) within a MATLAB Function block. Unlike Section 2, you must now use the estimated states $\hat{\boldsymbol{x}}(k)$ for feedback instead of the true states:

$$\boldsymbol{u}(k) = -\boldsymbol{F}\left(\hat{\boldsymbol{x}}(k) - \boldsymbol{x}^{\star}\right) + \boldsymbol{u}^{\star}. \tag{6}$$

For nonlinear systems, apply this observer-based controller to both the linearised and nonlinear models. Provide relevant plots to demonstrate that both the state estimation and the closed-loop system performance meet the Canvas requirements.

Remark 4. *Set* `run_observer = 1;` *in your* `main_SFC.m` *file run the SFC along with your observer.*

Remark 5. *To verify the dynamic response of the observer, create an initial observation error by initialising your observer with a different starting state vector than the physical system model, i.e.,* $\hat{\boldsymbol{x}}(0) \neq \boldsymbol{x}(0)$. *Provide relevant plots to show the observer convergence.*

References:

[1] R. P. Aguilera, *"Lecture Notes on State-Feedback Control"*, University of Technology Sydney, Australia, 2026.

[2] B. Messner, and D. Tilbury, *"Control Tutorials"*, University of Michigan, USA, link: http://ctms.engin.umich.edu/CTMS